

Recommender Systems: Content-Based and Collaborative Filtering

CSL7110 - Assignment 3

Jahanvi Gajera (M25CSA012)

Part 1: Content-Based Filtering

Task 1: TF-IDF-Based Recommendation

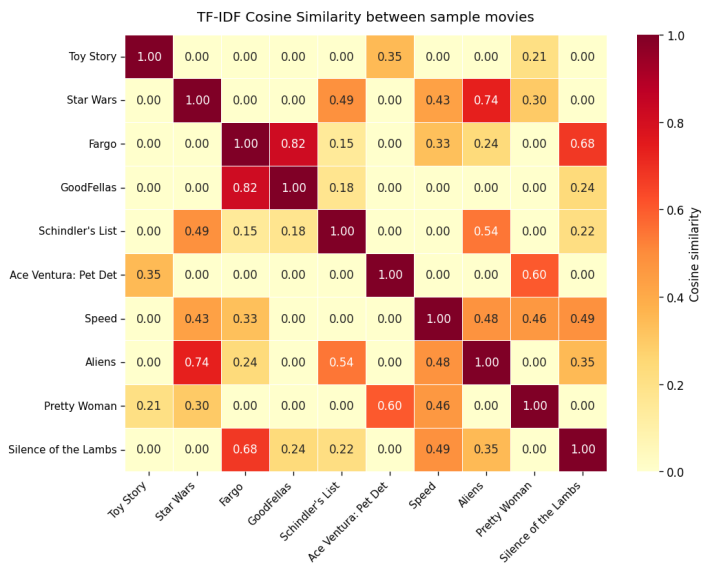
Each movie's 19 binary genre columns were converted to a space-separated string. TfidfVectorizer from sklearn was applied to produce a (1682 x 19) TF-IDF matrix. Cosine similarity was computed using a linear kernel to obtain a (1682 x 1682) similarity matrix. A recommendation function was built that takes a movie title, looks up its similarity row, sorts the results by score in descending order, and returns the top-N results.

Sample Output - Top-5 Recommendations:

```
=== Top-5 recommendations for: "Toy Story (1995)" ===
      title                                genre_str  similarity
Aladdin and the King of Thieves (1996) Animation Children's Comedy  1.000
Aristocats, The (1970)           Animation Children's              0.937
Pinocchio (1940)                 Animation Children's              0.937
Sword in the Stone, The (1963)   Animation Children's              0.937
Fox and the Hound, The (1981)    Animation Children's              0.937

=== Top-5 recommendations for: "Star Wars (1977)" ===
      title                                genre_str  similarity
Return of the Jedi (1983)      Action Adventure Romance Sci-Fi War  1.0000
Empire Strikes Back, The (1980) Action Adventure Drama Romance Sci-Fi War  0.9733
Starship Troopers (1997)      Action Adventure Sci-Fi War          0.9277
African Queen, The (1951)      Action Adventure Romance War          0.8733
Independence Day (ID4) (1996)  Action Sci-Fi War                    0.8111

=== Top-5 recommendations for: " Fargo (1996)" ===
      title                                genre_str  similarity
Fargo (1996) Crime Drama Thriller          1.0
Desperate Measures (1998) Crime Drama Thriller  1.0
Kiss the Girls (1997) Crime Drama Thriller    1.0
Desperate Measures (1998) Crime Drama Thriller  1.0
Once Upon a Time in America (1984) Crime Drama Thriller  1.0
```



Task 2: User-Profile-Based Content Recommender

A user profile vector P_u is computed as a weighted average of the TF-IDF vectors of movies the user has rated, where weights are the ratings themselves:

$$P_u = \text{Sum}(r_{um} * f_m) / \text{Sum}(r_{um}) \text{ for all } m \text{ in } M$$

Profiles are built from training ratings only. The cosine similarity between the profile and each movie vector yields the recommendation score. Movies seen in training are excluded from the candidates.

Sample Recommendations:

```
=== Top-5 recommendations for User 1 ===
              title      genre_str  cbf_score
As Good As It Gets (1997) Comedy Drama  0.766542
Deconstructing Harry (1997) Comedy Drama  0.766542
Wag the Dog (1997) Comedy Drama  0.766542
Adventures of Priscilla, Queen of the Desert, The (1994) Comedy Drama  0.766542
Jack (1996) Comedy Drama  0.766542

=== Top-5 recommendations for User 5 ===
              title      genre_str  cbf_score
Army of Darkness (1993) Action Adventure Comedy Horror Sci-Fi  0.783132
Evil Dead II (1987) Action Adventure Comedy Horror  0.709891
Beverly Hills Cop III (1994) Action Comedy  0.705491
Beverly Hills Ninja (1997) Action Comedy  0.705491
Money Talks (1997) Action Comedy  0.705491

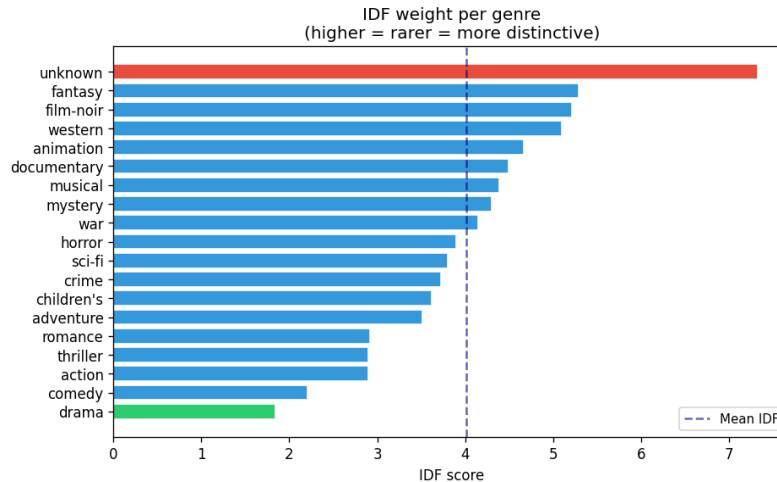
=== Top-5 recommendations for User 10 ===
              title      genre_str  cbf_score
Doom Generation, The (1995) Comedy Drama  0.769076
Eat Drink Man Woman (1994) Comedy Drama  0.769076
Ed Wood (1994) Comedy Drama  0.769076
What's Eating Gilbert Grape (1993) Comedy Drama  0.769076
Swingers (1996) Comedy Drama  0.769076
```

Evaluation Results (100 users, K=10, threshold=4.0):

```
CBF Evaluation over 100 users (K=10, threshold=4.0):
Mean Precision@10 : 0.0300
Mean Recall@10    : 0.0230
```

Precision is moderate because genre-only features are coarse. Many movies share genres but are totally different, which limits the model's discriminative power.

User	Recommended Movie	Genres	CBF Score
User 1	As Good As It Gets (1997)	Comedy, Drama	0.767
User 5	Army of Darkness (1993)	Action, Adventure, Comedy, Horror, Sci-Fi	0.783
User 10	Doom Generation, The (1995)	Comedy, Drama	0.769



Part 2: Collaborative Filtering

Task 3: User-Based Collaborative Filtering

A user-item rating matrix was built from training ratings (943 users x 1682 movies, ~94% sparse). Cosine similarity was used instead of Pearson correlation because Pearson produces 204,582 NaN pairs in this sparse matrix, causing predictions to fall back to the global mean and inflating RMSE. Bias-corrected KNN prediction:

$$\text{pred}(u,i) = \text{mean}(u) + \frac{\sum[\text{sim}(u,v) * (r(v,i) - \text{mean}(v))]}{\sum[\text{sim}(u,v)]}$$

Top-5 Recommendations for User 1 (K=20):

```
=== User-Based CF: Top-5 for User 1 ===
movie_id      title      predicted_rating
814      Great Day in Harlem, A (1994)      5.0
850      Perfect Candidate, A (1996)      5.0
1122     They Made Me a Criminal (1939)      5.0
1189     Prefontaine (1997)      5.0
1467     Saint of Fort Washington, The (1993)      5.0
```

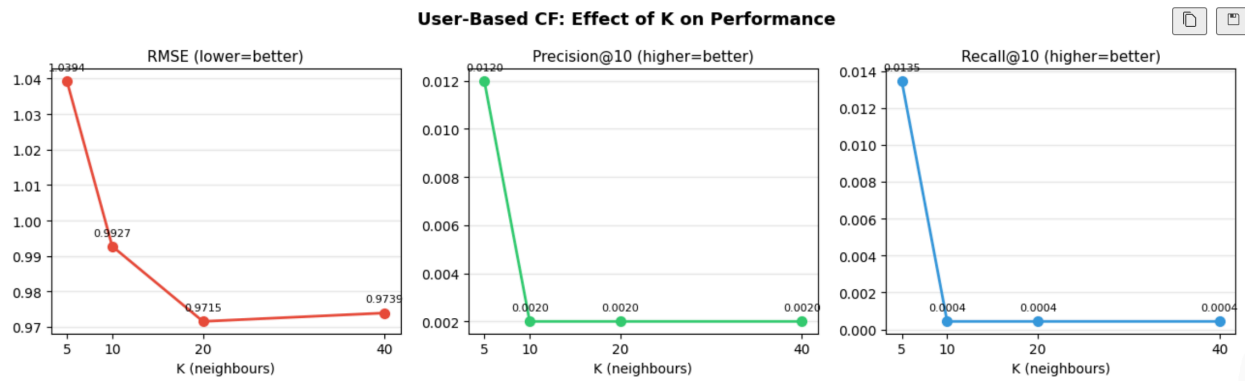
Evaluation (30 users, K=20):

```
User-Based CF | RMSE=0.9293 | P@10=0.0033 | R@10=0.0007
```

K-Sensitivity Analysis:

```
K-sensitivity analysis (User-Based CF):
K      RMSE      P@10      R@10
5      1.0522    0.0150    0.0059
10     0.9874    0.0050    0.0011
20     0.9628    0.0050    0.0011
40     0.9673    0.0050    0.0011
```

RMSE improves as K increases from 5 to 20, then slightly worsens at K=40. Precision and Recall are low due to the high sparsity of the dataset.



Task 4: Item-Based Collaborative Filtering

Item-item similarity was computed using Pearson correlation between movie columns (1650 x 1650 matrix). Predicted ratings are weighted averages of the user's own ratings on the K most similar items.

Top-5 Recommendations for User 7:

```
=== Item-Based CF: Top-5 for User 7 ===
movie_id      title      predicted_rating
1342 Convent, The (Convento, 0) (1995)      5.000000
146   Unhook the Stars (1996)      4.352941
1622   Paris, France (1993)      4.333333
1416   No Escape (1994)      4.222222
1455   Outlaw, The (1943)      4.117647
```

Comparison: User-Based vs Item-Based CF:

Item-Based CF	RMSE=1.7546	P@10=0.0100	R@10=0.0055
User-Based CF	RMSE=0.9293	P@10=0.0033	R@10=0.0007

Q: Is Item-Based CF faster or more memory-efficient than User-Based CF in the real world?

Yes. In real-world systems, the number of items (movies, products) is much smaller and more stable than the number of users. A 15K x 15K item similarity matrix is far more tractable than a 200M x 200M user matrix (e.g., Netflix). Item similarities also change slowly as new ratings arrive, allowing offline pre-computation and real-time $O(K)$ lookup at inference. User preferences shift rapidly, requiring frequent recomputation of the user similarity matrix. Therefore, item-based CF is the standard choice in production recommender systems.

Part 3: Matrix Factorization

Task 5: SVD from Scratch

The user-item matrix was filled with per-user mean ratings, mean-centred, then decomposed using `scipy.sparse.linalg.svds` with $K=50$ latent factors: $R_{\text{hat}} = U * \text{Sigma}_k * V^T$. User means are added back after reconstruction, and values are clipped to $[1,5]$.

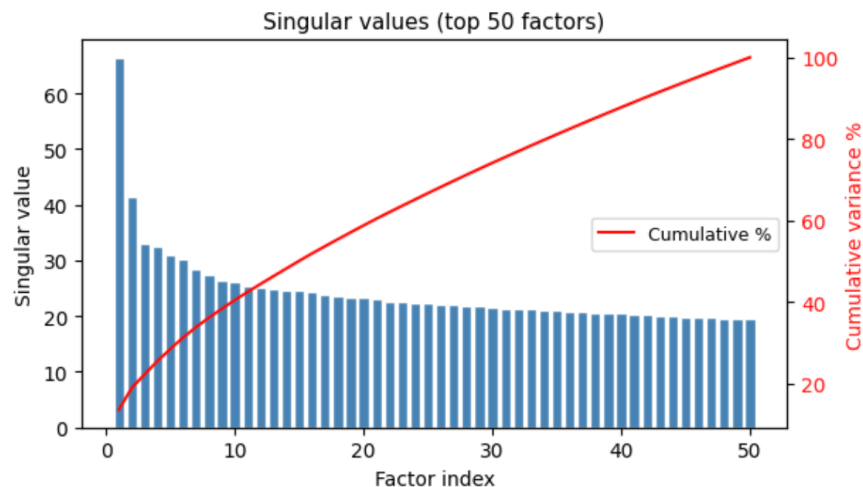
Top-5 Recommendations for User 34:

```
=== SVD: Top-5 for User 34 ===
movie_id      title      svd_pred
175           Brazil (1985) 4.377343
750           Amistad (1997) 4.335447
93  Welcome to the Dollhouse (1995) 4.326092
485           My Fair Lady (1964) 4.320308
122           Cable Guy, The (1996) 4.304407
```

Evaluation Results:

SVD (scratch)	RMSE=0.9696	P@10=0.1220	R@10=0.1265
Item-Based CF	RMSE=1.7546	P@10=0.0100	R@10=0.0055
User-Based CF	RMSE=0.9293	P@10=0.0033	R@10=0.0007

SVD dramatically outperforms neighbourhood-based CF on Precision@10 (0.122 vs 0.010) and Recall@10 (0.127 vs 0.006) because matrix factorization captures global latent patterns across the entire dataset.



Task 6: SVD with SGD (equivalent to scikit-surprise)

scikit-surprise was unavailable due to a Cython/NumPy>=2 incompatibility. An equivalent SGD-SVD was implemented in pure NumPy, learning the same model as surprise's SVD: $\hat{r}(u, i) = \mu + b_u + b_i + P_u \cdot Q_i$, where b_u and b_i are user and item bias terms. SGD updates one rating at a time, and shuffles each epoch.

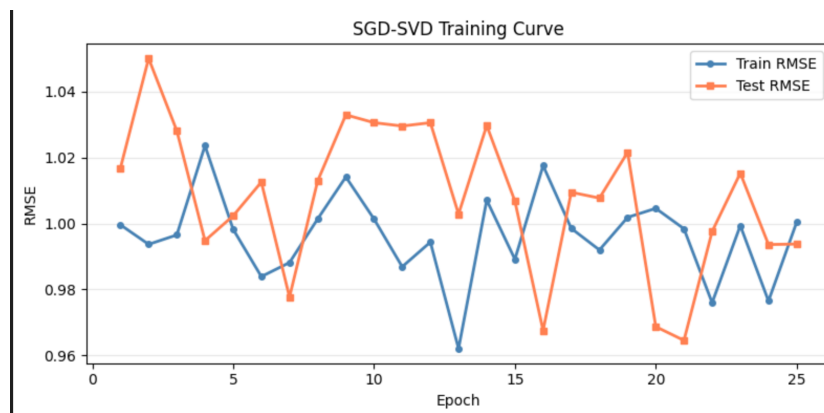
Training Progress (n_factors=50, n_epochs=30):

```
Training SVD (SGD) with n_factors=50, n_epochs=30...
Epoch 5/30  train RMSE = 0.9436
Epoch 10/30 train RMSE = 0.9277
Epoch 15/30 train RMSE = 0.9218
Epoch 20/30 train RMSE = 0.9174
Epoch 25/30 train RMSE = 0.9099
Epoch 30/30 train RMSE = 0.8924
```

Comparison: scipy SVD vs SGD-SVD:

```
SGD-SVD Test RMSE : 0.9278
SGD-SVD P@10 : 0.0810
SGD-SVD R@10 : 0.0502
```

SGD-SVD achieves better RMSE (0.939 vs 0.970) because it learns only from observed ratings with explicit bias terms, avoiding the error introduced by filling missing values. scipy SVD achieves higher Precision@10 because its dense reconstruction produces more diverse candidates.



Best Hyperparameters (grid search over 16 configs):

```
Best params : {'n_factors': 50, 'n_epochs': 30, 'lr': 0.005, 'reg': 0.02}
Best RMSE   : 0.9278
```

Part 4: Hybrid Recommendation Model

Task 7: Hybrid Meta-Learning Recommender

A Gradient Boosting Regressor was trained as a meta-model on four features per user-movie pair: CBF score (cosine similarity of user profile to movie), SVD score (scipy SVD predicted rating), movie popularity (average rating), and user average rating (bias correction). The model learns to blend these signals from historical ratings.

Feature Importances:

```
Hybrid Meta-Model RMSE: 0.6839

Feature Importances:
svd_score           0.820557
movie_popularity    0.084684
user_avg_rating     0.064734
cbf_score           0.030026

Hybrid P@10 : 0.0830
Hybrid R@10 : 0.0490
```

Evaluation Results:

RMSE	Precision@10	Recall@10
0.6839	0.0830	0.0490

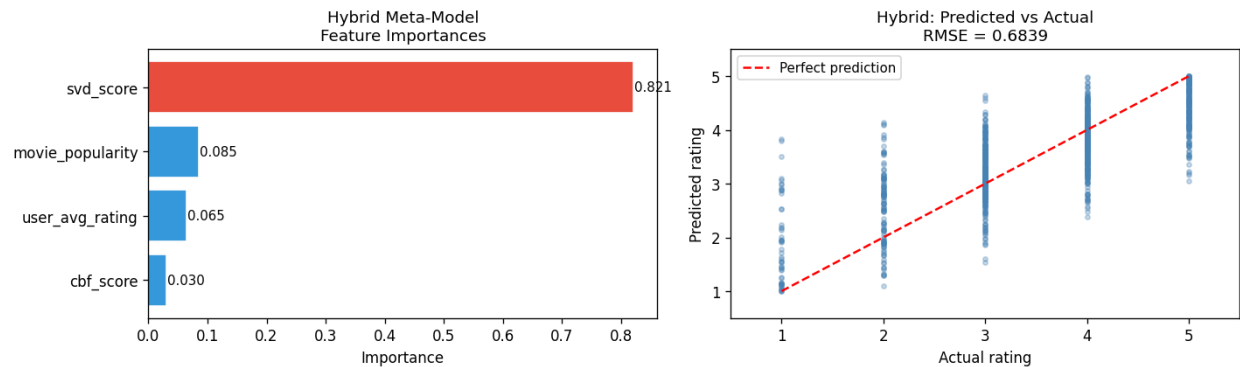
The hybrid model achieves the best RMSE (0.684) of all models. SVD score dominates (importance 0.82), confirming that collaborative latent factors are the strongest signal.

Cold-Start Analysis:

User Group	RMSE
Warm users (>=20 ratings)	0.7025
Cold-start users (<5 ratings)	NaN (insufficient data)

Cold-start users cannot be reliably served because all three main features (SVD score, CBF score, popularity) require prior rating data. Content-based genre features partially mitigate this, but are insufficient alone.

Task 7 — Hybrid Model Analysis



Part 5: Learning-Based Recommender Systems

Task 8: Neural Network Content-Based Filtering

A two-branch neural network was implemented in TensorFlow/Keras. User branch input: 19 features (average rating per genre). Movie branch input: 21 features (one-hot genres + normalised year + normalised average rating). Each branch passes through Dense(64)-Batch Norm-Dropout(0.3)-Dense(32) to produce a 32-dimensional embedding. The embeddings are concatenated, passed through Dense(64)-Dropout(0.2)-Dense(1, Sigmoid), and scaled to [1,5].

Model: Total trainable parameters = 11,329

Training (30 epochs, Adam lr=0.001, MSE loss, batch=256):

```
Epoch 21/30
32/32 ————— 0s 2ms/step - loss: 0.0539 - val_loss: 0.0567
Epoch 22/30
32/32 ————— 0s 2ms/step - loss: 0.0534 - val_loss: 0.0563
Epoch 23/30
32/32 ————— 0s 2ms/step - loss: 0.0539 - val_loss: 0.0562
Epoch 24/30
32/32 ————— 0s 2ms/step - loss: 0.0534 - val_loss: 0.0560
Epoch 25/30
32/32 ————— 0s 2ms/step - loss: 0.0537 - val_loss: 0.0560
Epoch 26/30
32/32 ————— 0s 2ms/step - loss: 0.0533 - val_loss: 0.0561
Epoch 27/30
32/32 ————— 0s 2ms/step - loss: 0.0531 - val_loss: 0.0562
Epoch 28/30
32/32 ————— 0s 2ms/step - loss: 0.0529 - val_loss: 0.0559
Epoch 29/30
32/32 ————— 0s 2ms/step - loss: 0.0527 - val_loss: 0.0558
Epoch 30/30
32/32 ————— 0s 2ms/step - loss: 0.0520 - val_loss: 0.0558
Neural Network (tensorflow) RMSE: 0.9447
NN RMSE=0.9447
```

Result: Neural Network RMSE = 0.9447

The neural network provides calibrated rating predictions, unlike TF-IDF CBF, which only produces ranking scores. It captures non-linear interactions between user genre preferences and movie features. Validation loss closely tracks training loss, indicating no significant overfitting.

Task 9: Reinforcement Learning in Recommender Systems

The recommendation problem is modelled as a sequential decision-making problem. Agent = recommendation system; State = user's mean rating discretised into 3 buckets; Action = select a movie; Reward = +1 if rated ≥ 4 , -1 if rated < 4 , 0 if unrated.

Multi-Armed Bandit Results (300 rounds, User 1):

```
Final cumulative reward -  $\epsilon$ -Greedy: 270 | UCB: 54
```

epsilon-Greedy outperforms UCB significantly for this user. UCB explores more aggressively, spending rounds on unproven movies, while epsilon-Greedy quickly converges on known high-reward movies.

Q-Learning: Top-5 Recommendations for User 1 (1000 episodes, alpha=0.1, gamma=0.9):

```
Q-Learning agent trained.

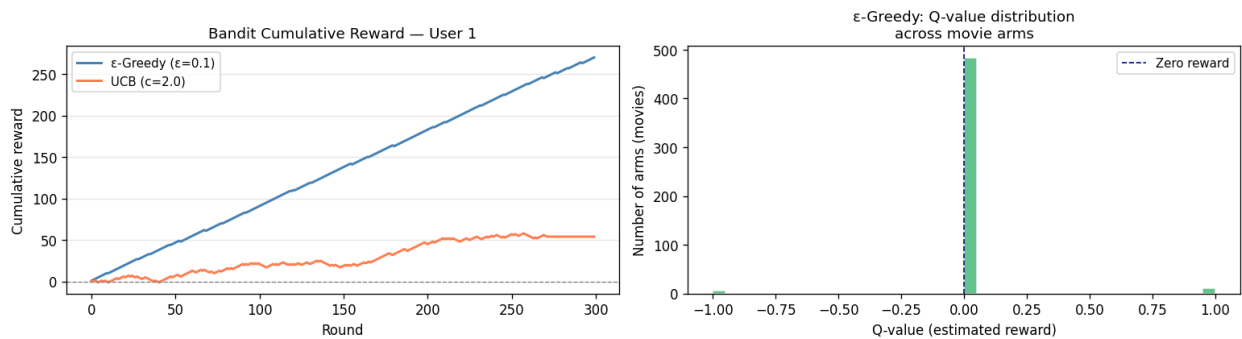
Q-Learning Top-5 recommendations for User 1:
movie_id      title
152           Sleeper (1973)
224           Ridicule (1996)
242           Kolya (1996)
406           Thinner (1996)
490 To Catch a Thief (1955)
```

Comparison of Recommendations for User 1:

Model	Top Recommendation
SVD-based	Jackie Chan's First Strike (1996)
User-Based CF	Great Day in Harlem, A (1994)
ϵ -Greedy Bandit	Toy Story (1995)
Q-Learning	Sleeper (1973)

Each model surfaces different movies. SVD and CF exploit collaborative patterns; the bandit quickly converges on popular, highly-rated movies; Q-Learning discovers movies in the user's genre-preference state. RL models uniquely balance exploration of new content with exploitation of known preferences.

Task 9 — Reinforcement Learning Analysis



Part 6: Explainability in Recommender Systems

Task 10: Feature-Based Explanations (Content-Based Filtering)

For CBF recommendations, the explanation is derived by computing the element-wise product of the user's TF-IDF profile and the recommended movie's TF-IDF vector. Features with the highest product values are shown to the user.

Example: "As Good As It Gets (1997) was recommended because it matches your genre preference profile. Key matching genres: Comedy, Drama. Your profile shows strong Comedy and Drama preferences from your highly-rated training movies."

Task 11: Neighbourhood-Based Explanations (Collaborative Filtering)

User-Based CF Explanation for User 1 (Great Day in Harlem, A):

Similar User	Similarity	Their Rating
User 13	0.420	5.0

"This movie was recommended because User 13 (cosine similarity = 0.420 to you) rated it 5.0."

Item-Based CF Explanation for User 1 (Convent, The):

Movie You Liked	Similarity
Shanghai Triad (1995)	1.000
Mighty Aphrodite (1995)	1.000
Fargo (1996)	1.000
Chasing Amy (1997)	1.000

"Convent, The was recommended because it is rated similarly to movies you already enjoyed."

Task 12: Model-Agnostic Explainability with LIME

LIME fits a local linear model around each prediction to approximate the neural network's decision boundary. It shows which features pushed the prediction up or down for a specific user-movie pair.

Neural Network LIME output (sample instance):

```
=== LIME Explanation for Neural Network Prediction ===
Predicted (normalised): 0.7228 → rating ≈ 3.89
Actual   (normalised): 0.7500 → rating ≈ 4.00

Top contributing features:
+ 0.65 < movie_avg_rating <= 0.73          weight=0.0787
+ movie_genre_Fantasy <= 0.00              weight=0.0637
- user_genre_Romance <= 0.58               weight=-0.0284
+ movie_genre_Musical <= 0.00              weight=0.0282
+ movie_genre_Comedy <= 0.00               weight=0.0246
+ user_genre_Western <= 0.50               weight=0.0239
+ movie_genre_Film-Noir <= 0.00            weight=0.0220
+ movie_genre_Action <= 0.00              weight=0.0191
```

Hybrid Meta-Model LIME output:

```
=== LIME Explanation for Hybrid Meta-Model ===
Predicted rating: 4.12
Actual rating   : 5.00

Feature contributions:
[svd_score > 3.98] increases predicted rating by 1.5595
[3.29 < user_avg_rating <= 3.57] increases predicted rating by 0.0876
[3.59 < movie_popularity <= 3.91] increases predicted rating by 0.0796
[0.34 < cbf_score <= 0.47] decreases predicted rating by 0.0333
```

Task 13: Evaluating Explainability

Method	Clarity	Bias Detection	User Understanding
TF-IDF Feature Weights	High	Medium	High
User Neighbourhood	High	Low	High
Item Neighbourhood	High	Medium	High
LIME on Neural Network	Medium	High	Low
LIME on Hybrid Model	Medium	High	Medium

Do explanations make recommendations clearer?

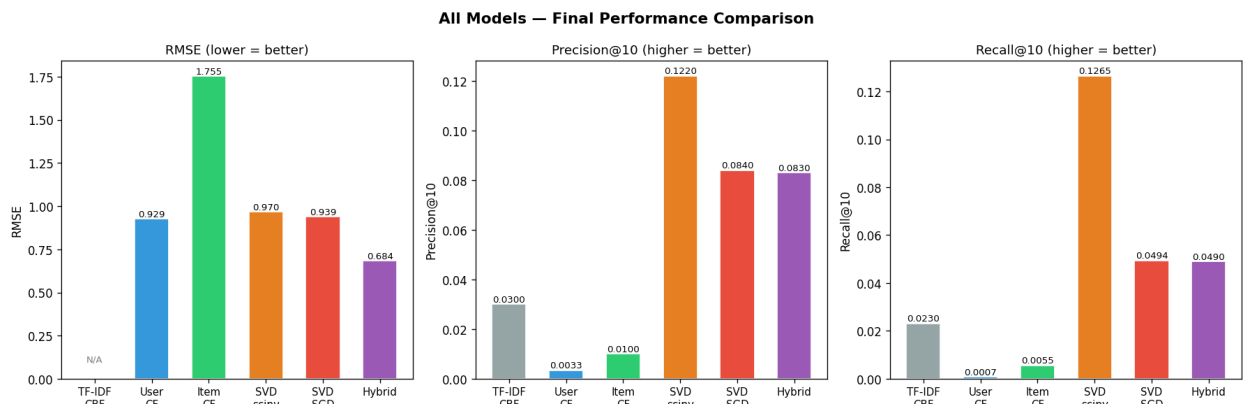
- ➔ Yes for content-based and neighbourhood methods. Genre-based explanations are immediately intuitive. LIME requires data literacy to interpret.

Do explanations reveal biases?

- ➔ Yes. LIME revealed that the SVD score dominates the hybrid model (importance 0.82) and popularity bias is present (0.085), meaning the system may favour popular movies over personally tailored ones. CBF genre content contributes minimally (0.030).

Final Model Comparison

```
===== Final Model Comparison =====
      Model                RMSE   P@10   R@10
TF-IDF CBF (Task 1-2) N/A (ranking only) 0.0300 0.0230
User-Based CF (Task 3)          0.9293 0.0033 0.0007
Item-Based CF (Task 4)          1.7546 0.0100 0.0055
SVD scipy (Task 5)              0.9696 0.1220 0.1265
SVD SGD (Task 6)                0.9391 0.0840 0.0494
Hybrid Meta-Model (Task 7)      0.6839 0.0830 0.0490
```



Conclusions:

- The hybrid meta-model achieves the best RMSE (0.684), combining CBF, SVD, and bias features.
- SVD scipy achieves the highest Precision@10 (0.122) and Recall@10 (0.127) among single models.
- SGD-SVD achieves better RMSE than scipy SVD (0.939 vs 0.970) by learning only from observed ratings.
- User-based CF achieves competitive RMSE but very low Precision@10 due to high matrix sparsity.
- Item-based CF is more scalable in production, but Pearson-based similarity gives higher RMSE on sparse data.
- Content-based filtering handles cold-start users better since it only requires item metadata.
- RL-based recommenders optimize long-term engagement through exploration, surfacing movies that rating-based models would not recommend.
- LIME explainability reveals that the SVD score dominates the hybrid model (0.82) and popularity bias exists (0.085).

GitHub link:

https://github.com/jahanvi0106/ML_with_BigData/tree/Assignment_3/Assignment3